

Opponent Action Classification & Modeling in the Pokerbot Engine

Executive Summary: We examine how the pokerbot's engine (RoundState/pkrbot-based) represents actions and how to update opponent models within a GTO-centric strategy. The engine's actions use *raise-to* semantics (the RaiseAction's `amount` is the new total pip, not an increment) ¹ ². A "Check" occurs when the opponent to-call cost is zero, whereas "Call" covers any positive `continue_cost` ³ ¹. By comparing consecutive RoundStates (pips and street), we infer opponent moves: e.g. if the opponent's pip increase equals the previous `continue_cost`, it was a call; if it exceeds, a raise. If both players' pips become equal and the street advanced, the last action was either a completed call or a second check (depending on whether a bet existed). Edge cases (all-in, minimum raises, capped pot) simply appear as large or bounded raises in pip differences ⁴ ². Notably, if `continue_cost ≥ stack`, raises are forbidden (only fold/call allowed) ⁵, and the RaiseBounds logic ensures minimum bet sizes and caps.

In alignment with DeepStack's continual re-solving design, **hero actions and chance (public card) events** warrant updates to our model (we update opponent counterfactual values and our range by removing impossible hands) ⁶. DeepStack explicitly **does not update on opponent actions** (it ignores which bet the opponent made) ⁶ – preserving equilibrium. To allow safe exploitation, we consider *soft updates*: track opponent action frequencies (fold/call/raise rates and bet-size histograms) and apply only mild biases. For example, GTO Wizard's "player profiles" approach adds small "action incentives" (e.g. "+5% call" for a calling-station opponent) which gently skew our strategy without fully re-solving to an exploitive best response ⁷. These incentives directly correspond to tracked frequencies and keep the strategy near-GTO.

We outline three modeling approaches: (A) **Strict GTO (no modeling)**, (B) **Profile-based incentives (static biases)**, and (C) **Adaptive frequency model (Bayesian updates)**. We compare their classification accuracy and computational cost, discuss pitfalls (misclassification leads to "wild, fragile" strategy shifts ⁸), and security/robustness (overfitting, information leakage). We find that a **hybrid: limited statistics + incentive adjustment** is the best fit – it uses only a few counters (fold/call/raise counts, bet-size bins) with decay, preserving determinism and low overhead. We recommend implementing a "Profile/Threshold" system (Approach B) with careful logging and rollback, as it strikes a balance between exploitability and robustness.

Approach 1: Strict GTO (DeepStack-style)

- **Method:** No opponent modeling beyond legal-actions inference. Follow a pure re-solving strategy: only update on **hero actions** (Bayesian update of our own range) and **chance deals** (eliminate impossible opponent hands) ⁶. Crucially, *do not* update on opponent bets or raises; treat every opponent decision as consistent with equilibrium.
- **Classification:** Minimal. We still must interpret RoundState to implement correct game logic, but do not feed opponent's last move into any model. Technically, classification still happens in code (RaiseAction vs CallAction vs CheckAction), but no statistics are recorded.

- **Tradeoffs:** Zero extra cost; preserves full Nash-exploitability guarantees. Memory: essentially just hold opponent's counterfactual value vector (no history). Computation: identical to baseline strategy. Exploitation: none. Observability: only engine logs (no model state).
- **Advantages:** Completely safe (no overfitting), straightforward to certify. Deterministic given fixed RNG seed (no external state). Compatible with DeepStack theory (updates only on own/chance actions) ⁶.
- **Disadvantages/Failure Modes:** Cannot exploit any opponent leak. Misclassification doesn't exist by design (since we ignore actions), but fails if opponent is significantly non-GTO. Over time this yields zero edge. Also, if any code mis-interprets actions (e.g. raise-to bug ¹), it may hide bugs.
- **Key considerations:** Must still implement *accurate* action semantics in the engine (raise-to vs raise-by, check vs call) to ensure game flow is correct ¹ ². Even with no model, erroneous classification will break game logic (e.g. wrong legal_actions). Logging should at least confirm action classification (fold/call/raise) is correct.

Approach 2: Profile-Based Action Incentives (Deterministic Exploitation)

- **Method:** Track simple action frequencies (fold/call/raise rates and bet sizing) and encode them as fixed "incentives" or threshold adjustments in our decision logic. For example, if an opponent is observed to call *more often* than baseline, we attach a small positive incentive to calling (treat calls as +x% EV); if they fold too often, we boost bluff frequency. This is analogous to GTO Wizard's profiling ⁷. We do *not* re-solve the entire game with a new distribution, but merely tweak decision thresholds or mixed-action probabilities.
- **Classification:** At each opponent action, compare previous and current `pips` and `street` to deduce the move (raise vs call vs check) as in Approach 1. Increment simple counters: e.g., number of opponent raises on river, fold-to-bet frequency, etc. Also record relative bet sizes into coarse bins. These counters are **lightweight features**: e.g. "opp 3-bet % = X% on this street."
- **Model Update:** Periodically (or after each hand) compute biased play tendencies. For example, if opp folds to flop continuation bet 80% of time, add a +Fold incentive so we bluff more. If opp calls large bets frequently, reduce folding margin. This can be done by adjusting numeric thresholds in `strategy.play` (e.g. lowering our fold threshold by 5% if opponent folds too much). The key is that adjustments are **small and additive** (e.g. $\pm 2-5\%$ to action probabilities) so that we stay near equilibrium.
- **Tradeoffs:** Very low computational cost. Stats are just integer counters; incentive adjustments are simple arithmetic at decision time. Memory is a handful of counters per street. This retains determinism (we can still use a fixed RNG seed for mixed choices) and fits the 60s/1GB budget easily. Slight exploit potential: if the opponent truly deviates, we will gradually exploit them via biased actions. However, because updates are gentle, exploitability remains low. Logging and observability are straightforward (just counters and applied adjustments).
- **Advantages:** Offers controlled exploitation. Robust against misclassification: only a few actions needed to nudge model. Compatible with small sample: each counter has a natural denominator (hands seen) and we can start biasing only once a minimum sample is reached. This **profiles opponent features safely** ⁷ (e.g. see Fig. [52], Fig. [56] for how profile incentives look). It also avoids the overhead of re-solving in strategy.
- **Disadvantages:** Approximate only. Aggressive opponents may not be fully exploited. If sample sizes are very low, noise may cause wrong biases. Overfitting risk: if we overreact to short-term variance, we may degrade EV. But this is mitigated by smoothing/decay and requiring thresholds before

updating. Example failure: misclassifying a raise-as-call repeatedly could wrongfully suggest an opponent is a “calling station,” causing suboptimal looser play. However, because biases are small, the worst-case effect is bounded (as GTO Wizard notes, “full-on” exploitation can produce “wild, fragile strategies,” so we keep biases tame) ⁸ .

- **Illustration:** We model each opponent type by action incentives. For instance, a “**Fish**” profile might have “prefers calling over raising.” In practice we encode this by +5% to calling (C+5) and -6% to checks, etc. Figure [52] below illustrates example archetypes (Fish, Maniac, etc.) with their biases ⁷ . The table in Figure [56] shows sample incentive values (“C+4, X-6” means +4% call, -6% check incentive) for each profile. These incentives map directly to our tracked frequencies and translate into slightly adjusted action mixes in strategy ⁷ .

⁷ Profiles can be encoded by small “action incentives” on check/call/fold/bet to reflect an opponent’s tendencies. GTO Wizard, for example, models a “calling station” by adding +6% to calling and -3% to bet, making calls slightly more attractive ⁷ .



Figure: Example opponent archetypes (“Fish”, “Maniac”, etc.) labeled by their action biases, as used in GTO Wizard profiles ⁷ .

⁷ Each profile’s incentives translate to fold/call/raise frequencies. For example, Figure [56] lists incentives used for common types: a “Fish” has Call+4%, Check-6%, a “Maniac” has Bet+6%, Call+3%, Fold-6%, etc. These numeric bonuses adjust our decision thresholds to exploit that profile ⁷ .

Profile	Style	Incentives
GTO	No incentives (exploiting or equilibrium)	(none)
Fish	Likes to donk bet. Prefers calling over raising	C+4, X-6
Calling Station	Passive and sticky	B-3, C+6, X+2
Glass Cannon	Attacks aggressively, defends cautiously	B+4, F+5, X+1
Maniac	Very aggressive and sticky	B+6, C+3, F-6
Nit	Passive and folds to aggression	B-3, F+7, X+2
TAG Reg	Overall solid player, slightly conservative	C-2, F+2, X+2
LAG Reg	A tough, splashy player with a strong foundation	B+3, F-3

Figure: Sample profile incentives (Bet/B, Call/C, Fold/F, Check/X) for common player types. These fixed offsets effectively modify our action probabilities and are tied to the tracked fold/call/raise rates ⁷.

Approach 3: Adaptive Frequency Model (Bayesian Update)

- **Method:** Maintain a probabilistic model of the opponent's hand range and update it with each observed action. On each opponent bet/call, use Bayes' rule to refine the opponent's range or counterfactual values, as in traditional CFR/BPM/Opponent-Modeling literature ⁶. For example, if an opponent bets strongly on the flop, downweight the subrange of hands that would never lead to that bet (using an assumed prior strategy or reaching probabilities). Then re-solve the game from that public state using the updated opponent range.
- **Classification:** Critical. We must correctly classify every opponent action (fold/call/raise size) to apply the right likelihood update. This means carefully handling multi-raise sequences, all-in caps, min-raises, etc., so that our Bayes update reflects the true event. For instance, distinguishing a check-raise from a limp-call-raise cycle by looking at the pip differences and street progression. If classification fails (e.g. treat a small raise as no raise), our range update is wrong, causing strategy drift.
- **Model Update:** On **hero actions**, update opponent counterfactual values from the solve as DeepStack does (keep values from chosen branch) ⁶. On **chance (card deal)**, eliminate impossible opponent hands ⁶. On **opponent actions**, unlike DeepStack we *do* update: use the action likelihood to adjust opponent's range distribution. However, to preserve low exploitability, we must weight this gently. One approach is to use a "mixture" of the original GTO range and the Bayesian-updated range (e.g. blend by a factor $\alpha \ll 1$) or to update only for hands in mixed strategies (following "safe exploitation" rules). Ganzfried & Sandholm's safe-exploitation work suggests ways to exploit while bounding exploitability ⁹.
- **Tradeoffs:** High accuracy potential (can eventually approximate best-response to an exploitable opponent) but heavy. Each decision now may entail a CFR or re-solve step (even if depth-limited), which may exceed our CPU budget. Moreover, memory to store and update ranges or counterfactual values per possible hand can be large. It also complicates determinism (re-solving can introduce random tie-breaking) and risks overfitting if not carefully regularized.

- **Advantages:** Maximally exploits opponent leaks if done well. Can adapt to adversaries in principle. If combined with safe bounds, it still ensures some worst-case guarantee ¹⁰ ⁸ . Observability: rich logging can track entire range shifts.
- **Disadvantages/Failure Modes:** If classification errs or the opponent strategy changes mid-hand, updates may be grossly wrong. Overfitting: small sample Bayes updates can swing the model far from reality. EV impact: misclassification leads to suboptimal plays (e.g. if we think an opponent raised when he actually called, we may bluff into a strong range). GTO Wizard warns that “max exploit” strategies can produce “wild, fragile” behavior ⁸ if the model is wrong. This approach also risks leaking private information: if our model learns too strongly, the bot’s actions could inadvertently reveal our own strategy or range in predictable ways.
- **Implementation Cost:** This is the most complex. Requires modular “opponent solver” code, careful weight normalization, and possibly stochastic elements. We must also set a **decay schedule** and minimum-sample threshold: e.g. only update the range if $\geq N$ observations, and fade old data with an exponential decay (half-life on the order of a few hundred hands) to adapt to non-stationary opponents. These safeguards (common in practice) prevent early swings from tiny samples.

Failure Cases & Impact: In any approach, misclassification is the biggest pitfall. For example, confusing a small raise for a call (pip increase misread) causes us to attribute wrong strength, potentially shifting billions of chip-EV over time. As GTO Wizard notes, extreme exploitation (driven by naive stats) can make our strategy “fragile” ⁸ , costing EV if the opponent adjusts. Opponent modeling also introduces privacy risk: if we log too much detail (like unique hand histories), we might “leak” sensitive patterns in logs or allow training set attacks. Compatibility/versioning: any model parameters or thresholds should be versioned; rollbacks must be easy if a new exploit tweak backfires. Observability: we recommend extensive logging of classification decisions (e.g. log “CLASSIFIED_ACTION=Raise(1500)” each time) and periodic dump of opponent feature stats. At minimum, log aggregate fold/call/raise rates by street each epoch so we can audit the model.

Approach Comparison

Feature/Approach	(A) GTO-Only (DeepStack)	(B) Profile Incentives (Static Bias)	(C) Bayesian Update (Dynamic)
Classification Needed	Only for engine correctness (legal actions) ¹ . No stats used.	Basic: infer opponent raises/calls/checks to increment counters. Simple pip-diff logic ² .	Full: infer every action precisely, including sizing, in order to update probabilities.
Update On Opponent Acts	Never (range unchanged) ⁶	No direct range update; only counters/frequencies adjusted.	Yes, with Bayesian belief update (subject to safety constraints) ⁶ .
Update On Hero/Chance	Yes (DeepStack style) ⁶	Yes (we can still do DeepStack-style range updates on these)	Yes (as in DeepStack) + possibly range re-solve

Feature/Approach	(A) GTO-Only (DeepStack)	(B) Profile Incentives (Static Bias)	(C) Bayesian Update (Dynamic)
Key Features Tracked	None (or trivial uniform prior)	Fold/call/raise counts by street, bet-size bins, CP (continuation bet) stats ⁷ .	All of (B) + full posteriors of hand equities (counterfactual values).
Exploitative Power	Minimal (strict equilibrium)	Moderate (small controlled biases)	High (full best-response if accurate)
Complexity & Latency	Lowest (just baseline strategy code).	Low (few counters + simple math at decision).	Highest (requires re-solving or CFR on the fly).
Sample Requirements	None.	Moderate – need enough data to trust a bias.	High – need lots of data to accurately update range; guard by thresholds.
Robustness (Low Exploitability)	Excellent (provably close to Nash) ⁶	Good (biases are small by design).	Risky (must enforce safe/exploit bounds, else can be overfit).

Security, Privacy, and Observability

Across all approaches, we must be cautious not to overfit short-term “bad beats” or reveal private strategy. Stored opponent stats should use only public information (actions, public cards), never their private hand. Logging should be sanitized (e.g. no saving board/hand history beyond what engine provides) to avoid leaking secrets. To limit information leakage, we suggest **decoupling** the model from the solver: rather than feeding raw stats into the internal solver, use only the coarse incentives/thresholds. All opponent-model code should run off the main critical path (per-decision), with effects small enough that failures degrade gracefully.

Observability requires minimal runtime overhead. For example, simply printing (or saving) a summary JSON each game-end with fields like `opp_fold%_FLOP`, `opp_call%_TURN`, `avg_raise_size_PFR`, etc. – similar to the plan’s harness metrics ¹¹. We should also validate action classification by logging every opponent action’s interpreted type (fold/call/raise) and detected size. This lets us audit classification correctness offline.

We must also prepare **rollback strategies**. If a new modeling tweak causes worse self-play results, we need to disable it. The code should allow switching off adaptation (e.g. environment variable) to revert to plain GTO. Version the opponent-model parameters (decay rates, thresholds) alongside agent code.

Recommendation

Given the t3.micro constraints (60s/CPU, 1GB memory) and the need for determinism, we recommend **Approach 2: Profile-based incentives** as the best-fit. It leverages a few hand-picked features that offer safe exploitation without heavy computation. Specifically: track fold/call/raise frequencies and bet sizes on each street, apply an **exponential decay** (e.g. half-life ~200 hands) so recent behavior matters more, and only

adjust strategy once a reasonable sample (e.g. ≥ 50 actions) is observed. Use small additive bonuses ($\pm 2-6\%$) to adjust mixed decisions (e.g. increase calling weight if opponent folds too much) ⁷. This approach is easy to code (fits in `strategy.py` after classification), deterministic (we can still seed randomness per hand), and yields measurable benefits against exploitable opponents with negligible risk. The recommended parameterization is: - Decay factor: use geometric decay per hand or round (for instance multiply old counts by 0.98 each new hand). - Thresholds: require at least $N=20$ actions of a type before biasing. - Incentive magnitudes: cap at a few percentage points.

Implementation Notes: Update the opponent model only in `player.py` before calling `strategy.play(view)`, as per architecture ¹². Derive opponent action type by comparing `view.pips` and `view.to_call` across the previous RoundState. Then update the chosen counters. In `strategy.play`, add a small bias term to action utilities or probabilities based on these counters. Maintain deterministic RNG seeds (hash on hand ID/seat/action-index) for any mixing ¹³. Ensure legal-action sanity (if a raise advice is illegal, degrade to call or check, never directly to fold ¹⁴).

Checklist for Implementation

- **Action Semantics:** Confirm engine uses *raise-to* semantics. In `RoundState.proceed`, RaiseAction's `amount` should be treated as total bet (as seen in skeleton code ²). Ensure all code uses this convention consistently (our logs or harness should flag if a "min_raise" is never reached, a common bug ¹).
- **Action Classification:** In `player.py`, when it's the opponent's turn, infer action by comparing the previous `view` and new `view`:
 - If opponent's pip increased above hero's pip, it's a *raise*.
 - If pips equalize and `continue_cost` just became 0, it's a *call*.
 - If pips unchanged and `continue_cost=0`, it's a *check*.
 - If the game ends, it's a *fold*.
 Add robust checks around street changes: if the street advanced, the last action was either a call or a second check (infer which by whether any bet had occurred).
- **Update Triggers:** Only update counters for **opponent's actions** (and optionally at showdown for discard info, see below). Update our internal range or counterfactual values only on *hero* and *chance* events ⁶.
- **Model Features:** Maintain lightweight stats: for each street, count (folds vs hero bet), (calls vs hero bet), (raises vs hero bet), and a histogram of raise sizes (bucketed as small/medium/large relative to pot). Also track pre-flop preflop 3-bet, c-bet, etc. Do *not* store full action history.
- **Incentive Logic:** In `strategy.py`, after computing GTO-action probabilities, adjust them slightly: e.g. if `opp_fold_rate` on flop > expectation, add a small bonus to bluff actions; if `opp_call_rate` is high, increase our calling propensity. Always renormalize probabilities. Use fixed rules or interpolated incentives (like Table [56]). Never drop a raise-favored action to fold due to illegal sizing; instead degrade *downwards* (as recommended) ¹⁴.
- **Opponent Discard:** The opponent's post-flop discard can be treated like public info: eliminate any opponent hands containing that card from our range (zero probability) – but downweight this update since discard is decision-driven. For example, use it akin to a "chance" update but multiply the range by ≤ 1 (so as not to overweight).
- **Decay & Thresholds:** Implement exponential decay on all counters (e.g. multiply counts by 0.99 each new hand) to forget very old data. Only apply any incentive if the underlying sample size exceeds a threshold (e.g. 30 observations) to avoid reacting to noise.

- **Logging/Observability:** Log each opponent action classification (e.g. "OPP_ACTION=Raise(%amount%)"). Periodically (e.g. each round or end-of-hand) log summary stats in a parseable format (JSON) with fields like `flop_opp_fold_rate`, `turn_opp_call_rate`, `avg_raise_pct`, etc., as suggested in [28]. This allows off-line verification of classification accuracy and model effect.
- **Security/Privacy:** Do not log private hand or deck info. Only log public cards, stack sizes, and actions. Avoid storing raw sequences of opponent cards – use only aggregate stats.

References

- Action semantics and engine logic: official RoundState code and design docs ¹ ² explain *raise-to* amounts and check vs call.
- DeepStack continual re-solving update rules ⁶ provide the baseline “update on hero/chance, ignore opponent action” policy.
- Safe exploitation literature: Norris & Watson (2013) on adding statistical exploitation without raising exploitability ¹⁵ ; Ganzfried & Sandholm (2015) on safe best-response guarantees ⁹ .
- GTO Wizard profiles (“action incentives”) as an applied model of opponent features ⁷ ⁸ .
- Poker AI engineering best practices (as discussed in team design docs ¹ ¹⁴) for legality handling and logging.

¹ ¹¹ ¹² ¹³ ¹⁴ `what_runs_when.md`

https://github.com/nyx-haile/pokerbots/blob/db8e957d21ff2636df2cbfec35857580ee027b38/design/what_runs_when.md

² ³ ⁴ ⁵ `roundstate.py`

<https://github.com/vihank/poker-engine-2024/blob/a8b914435be1fe955f04e22bd443a348a737f873/engine/roundstate.py>

⁶ `DeepStack.pdf`

<file:///file-K9UDT85rtD1y2YEWkhhbZye>

⁷ ⁸ Profiles Explained: Modeling Exploitable Opponents | GTO Wizard

https://blog.gtowizard.com/profiles_explained_modeling_exploitable_opponents/

⁹ ¹⁰ ¹⁵ Opponent Modeling and Exploitation in Poker Using Evolved Recurrent Neural Networks

<https://nn.cs.utexas.edu/downloads/papers/li.phd18.pdf>